WEB AUTOMATION GRADUATION PROJECT

Test Automation Plan & Project Requirements Document

Track: Digital Egypt Pioneers Initiative (DEPI) – Software Testing Track
Application Under Test (AUT): Your Store (`awesomeqa.com`)

1. Executive Summary & Project Overview

This document defines the Test Automation Plan, functional requirements, framework architecture, and team execution strategy for the **Web Automation Graduation Project**. The primary objective is to build a scalable, maintainable, and robust automated testing framework using **Selenium WebDriver** and **Java** to verify core e-commerce user flows on **Your Store (`awesomeqa.com`)**, expand test coverage with 20 additional test scenarios, and generate an interactive **Extent Report** for test execution analysis.

1.1 Core Project Deliverables

- **Core Test Suite Automation:** Automate the predefined core test cases specified in the project requirements (covering all 12 functional scenarios listed in the specification).
- **Extended Test Coverage:** Design 20 additional test cases covering critical e-commerce workflows, edge cases, and negative scenarios, and automate them completely.
- **Execution Reporting:** Integrate and generate a comprehensive HTML **Extent Report** documenting test execution results, step logs, and failure screenshots.

---

2. Scope of Testing & Functional Requirements Breakdown

The total scope consists of 32 automated test cases (12 predefined core scenarios + 20 newly designed scenarios), categorized into 4 functional modules to streamline parallel development across the 5 team members.

2.1 Predefined Core Test Cases (12 Test Cases)

Test Case ID	Module	Test Case Description	Scenario Type
TC-01	User Authentication	Verify Registration Functionality with valid and invalid inputs	Positive & Negative
TC-02	User Authentication	Verify Login Functionality with valid and invalid credentials	Positive & Negative
TC-03	User Authentication	Verify Forget Password Functionality and email validation	Positive & Negative
TC-04	Search & Localization	Verify that logged-in users can search for any product	Positive
TC-05	Search & Localization	Verify that logged-in users can switch between currencies (US Dollar - Euro)	Positive
TC-06	Catalog Navigation	Verify selecting Main and Sub Categories randomly	Positive
TC-07	Catalog Navigation	Verify hovering over Main and Sub Categories randomly	Positive
TC-08	Catalog Navigation	Verify that logged-in users can select and browse different categories	Positive
TC-09	Shopping Lists	Verify adding items to the Wishlist and validating Wishlist contents	Positive
TC-10	Shopping Lists	Verify adding items to the Compare List and verifying comparison table	Positive
TC-11	Shopping Cart	Verify adding items to the Shopping Cart and validating cart badge/total	Positive
TC-12	Order Checkout	Verify creating an End-to-End order with a successful payment	Positive (E2E)

2.2 Additional Designed Test Cases Scope (20 Test Cases)

To achieve comprehensive test coverage, the team will design and automate 20 additional test cases distributed across the following areas:

Shopping Cart Management (5 TCs)

- Update product quantity in the shopping cart and verify dynamic price recalculation.
- Remove an item from the shopping cart and verify empty cart message.
- Apply an invalid coupon code and verify error alert message.
- Apply an invalid gift certificate and verify validation message.
- Verify shipping estimate calculation using country and region dropdowns.

Product Details & Catalog Filtering (5 TCs)

- Sort products on category pages by Price (Low to High / High to Low) and Name (A–Z).
- Submit a product review (valid and invalid rating/character length).
- Verify product image gallery modal/lightbox navigation on the product page.
- Search for a non-existent product and verify the "No product matches" message.
- Verify product availability status and tax price display on the product details page.

User Account & Address Book Management (5 TCs)

- Update user profile information (First Name, Last Name, Telephone) in My Account.
- Change account password and verify login with the updated password.
- Add a new shipping/billing address in the Address Book.
- Delete or edit an existing address in the Address Book.
- Verify Order History and Return Request submission from the user dashboard.

UI, Footer & General Store Validations (5 TCs)

- Validate the "Contact Us" form submission with valid and invalid email formats.
- Verify Brand/Manufacturer filtering from the footer link ("Brands").
- Verify removing an item from the Wishlist and transferring an item from Wishlist to Cart.
- Verify checkout failure when mandatory billing details or Terms & Conditions checkbox are skipped.
- Verify switching currency to Pound Sterling (£) and validating updated product prices across pages.

---

3. Test Automation Framework Architecture

3.1 Tools & Technology Stack

Component	Selected Tool / Technology	Purpose & Justification
Programming Language	Java (JDK 17+)	Industry standard, strong OOP support for Page Object Model
Web Automation Tool	Selenium WebDriver (v4.x)	Direct browser interaction across Chrome, Edge, and Firefox
Test Runner & Assertions	TestNG	Supports annotations, parallel execution, grouping, and `@DataProvider`
Build & Dependency Tool	Apache Maven (`pom.xml`)	Centralizes dependency management across all 5 team members
Design Pattern	Page Object Model (POM) + Data-Driven (DDT)	Separates WebElements from test scripts to prevent Git merge conflicts
Reporting Library	ExtentReports (AventStack v5)	Generates rich interactive HTML execution reports with screenshots
Version Control	Git & GitHub	Collaborative code management, branching, and pull request reviews

3.2 Framework Directory Structure

- **`src/main/java/pages/` (Page Object Layer):** Contains dedicated classes for each webpage (`HomePage.java`, `RegisterPage.java`, `LoginPage.java`, `SearchResultsPage.java`, `ProductPage.java`, `WishlistPage.java`, `ComparePage.java`, `ShoppingCartPage.java`, `CheckoutPage.java`). Stores private `By` locators and public action methods (e.g., `enterCredentials()`, `selectCurrency()`, `hoverOverRandomCategory()`).

- **`src/test/java/base/` (Base Test Layer):** Contains `BaseTest.java` handling WebDriver initialization (`@BeforeMethod`), implicit/explicit wait configurations (`WebDriverWait`), configuration property loading, and browser teardown (`@AfterMethod`).
- **`src/test/java/tests/` (Test Execution Layer):** Contains modular TestNG test classes (`RegistrationTests.java`, `LoginTests.java`, `CatalogNavigationTests.java`, `CartAndCheckoutTests.java`). Uses TestNG assertions (`Assert.assertEquals`, `Assert.assertTrue`, `SoftAssert`) to validate expected vs. actual outcomes.
- **`src/test/java/utilities/` (Utilities & Reporting Layer):** `ExtentReportManager.java` & `TestListener.java` (implementing `ITestListener`) automatically log test execution statuses (Pass/Fail/Skip) and attach failure screenshots. `JsonDataReader.java` / `ExcelReader.java` feeds test data into `@DataProvider` methods for Positive and Negative scenarios.
- **`src/test/resources/testdata/`:** External JSON/Excel files holding registration credentials, login datasets, search keywords, and checkout billing details.
- **`testng.xml`:** Master XML suite file configuring test listeners and orchestrating full regression execution.

3.3 Step-by-Step Workflow to Create Each Automation Script

1. **Element Inspection:** Open the target store (`awesomeqa.com`), inspect target web elements via Chrome DevTools, and craft reliable locators (prioritizing `id`, `name`, `cssSelector`, and relative `xpath`).
2. **Page Class Implementation:** Define element locators and write clean, reusable interaction methods inside the corresponding Page Object class using explicit waits (`ExpectedConditions`) to avoid synchronization issues.
3. **Test Script Creation:** Create a `@Test` method in the corresponding test class, invoke the Page Object methods sequentially, and verify the result using TestNG assertions.
4. **Data Parameterization:** Connect Positive and Negative test cases (such as Registration and Login) to a `@DataProvider` to run multiple datasets against the same test method without code duplication.
5. **Listener & Extent Report Verification:** Run the test via `testng.xml` and inspect the generated `ExtentReport.html` file to ensure all steps and assertions are accurately logged.

---

4. Team Role Allocation & Project Schedule

4.1 Work Distribution Matrix

Team Member	Assigned Core Test Cases	Assigned Additional Test Cases	Shared Framework Role
Member 1	TC-01 (Registration Pos/Neg), TC-02 (Login Pos/Neg)	4 Test Cases (Account & Password Management)	Maven Setup, `BaseTest`, GitHub Repo Admin
Member 2	TC-03 (Forget Password), TC-04 (Product Search), TC-05 (Currency Switch)	4 Test Cases (Search Edge Cases & Localization)	`ExtentReportManager` & `TestListener` Setup
Member 3	TC-06 (Random Category Select), TC-07 (Random Category Hover), TC-08 (Select Categories)	4 Test Cases (Product Sorting, Filtering & Reviews)	CommonWait Utils & Random Helper Methods
Member 4	TC-09 (Add to Wishlist), TC-10 (Add to Compare List)	4 Test Cases (Wishlist/Compare Removal & Footer/Contact Us)	Data-Driven Reader (`JSON`/`Excel` Utility)
Member 5	TC-11 (Add to Shopping Cart), TC-12 (End-to-End Order & Payment)	4 Test Cases (Cart Quantity, Coupons & Checkout Validations)	`testng.xml` Suite Configuration & CI Execution

4.2 Execution Milestones

- **1. Milestone 1 – Planning & Test Case Design:** Finalize the Test Automation Plan document and write all 32 manual test cases (12 core + 20 additional) in a shared Google Sheet.
- **2. Milestone 2 – Base Framework & Environment Setup:** Initialize the Maven POM framework, configure Extent Reports, push the base project to GitHub, and verify local setup across all 5 machines.
- **3. Milestone 3 – Page Objects & Test Scripting:** Develop Page Classes and automate all 32 test cases on separate Git feature branches (`feature/auth-tests`, `feature/catalog-tests`, etc.).
- **4. Milestone 4 – Code Review, Integration & Final Report:** Merge all Pull Requests into the `main` branch, execute the full 32-test regression suite via `testng.xml`, resolve any flaky tests, and export the final **Extent Report** HTML deliverable.
